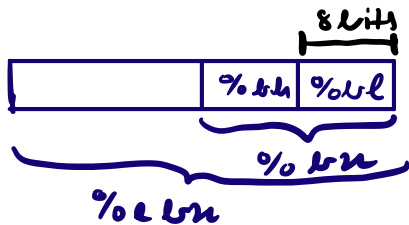


4) 4.1)



$$721 : 256 = 2 \times 256 + 209$$

Logo bh vai armazenar o n° 2
e bl 209

4.2) a- i) 110_{16}

ii) AB_{16}

iii) 118_{16}

iv) FF_{16}

v) AB_{16}

$$\begin{aligned} \text{vi) Endereço} &= \%ecn + (\%ecb \times 1) + 9 \\ &= 110 + 3 + 9 \\ &= 110 + \underbrace{12}_{\text{"C"}} \\ &= 11C \end{aligned}$$

Valor é 55_{16}

$$\text{vii) } 280 : 16 = 17 \times 16 + 8$$

$$17 : 16 = 1 \times 16 + 1$$

$$1 : 16 = 0 \times 16 + 1$$

$$280_{10} = 118_{16}$$

$$\begin{aligned} \text{Endereço} &= \%ecn + (\%ecb \times 1) + 118 \\ &= 1 + 3 + 118 \\ &= 1 + \underbrace{12}_{\text{"C"}} = 11C \end{aligned}$$

Valor é 55_{16}

$$\begin{aligned}
 \text{viii) Endereço} &= 0 + (\%edx \times 8) + 0xFC \\
 &= 24_{10} + 0xFC \\
 &= 18 + FC \\
 &= 1620 \\
 &= 174 \\
 &= 114
 \end{aligned}$$

O Valor é AB

$$\begin{aligned}
 \text{ix) Endereço} &= \%eax + (\%ebx \times 1) + 2 \\
 &= 110 + A + 2 \\
 &= 110 + C \\
 &= 11C
 \end{aligned}$$

O Valor é 55

↳ **addl $\%eax, \%ebx$**

$$110 + A = 11A \text{ guardado em } \%ebx$$

addl $(\%eax), \%ecx$

$$FF + 1 = 100 \text{ guardado em } \%ecx$$

subl $4(\%eax), \%edx$

$$3 - (AB)$$

$$3_{10} - 179_{10}$$

$$= -168_{10}$$

↓

Passar a comp. 2.

$$168 : 16 = 10 \times 16 + 8$$

$$10 : 16 = 0 \times 16 + 10$$

$$\text{Logo } 168_{10} = A8_{16}$$

Como temos um registro de 32 bits
vai ser representado:

000000A8

↓

i) inverter (tira a 'F', o valor)

$$F - 0 = F$$

$$F - \underbrace{A}_{\text{"10"}^4} = 15 - 10 = 5$$

$$F - 8 = 15 - 8 = 7$$

Logo temos

FFFFFFFF57

ii) Somar 1

FFFFFFFF58 guardado em %edx

andl \$43, (%eax, %edx, 4)

$$110 + (3 \times 4)$$

$$= 11 + \underbrace{12}_{\text{"C"}}$$

$$= 11C$$

55 AND 43

$$5 = 0101_2$$

$$4 = 0100_2$$

$$3 = 0011_2$$

$$0101 \ 0101 \ (5 \ 5)$$

$$0100 \ 0011 \ (4 \ 3)$$

$$\begin{array}{r} 0100 \ 0001 \\ \hline \underbrace{0100}_4 \ \underbrace{0001}_1 \end{array}$$

Logo é armazenado 41 em 11C

decl %edx

$3 - 1 = 2$ guardado em %edx

incl 8(%eax)

Endereço = $110 + 8 = 118$

$13 + 1 = 14$ guardado em memória de endereço 118

imull %eax, %ebx

$110 \times A$

= $A \times A$ guardado em %ebx

sal \$2, %ebx

O valor em ebx é 1010

sal \$2 desloca 2 bits para a esquerda

101000

28 em hexadecimal

4.3) Endereço Alvo = Endereço da próxima instrução + offset

(A) A instrução começa em 40F780 e ocupa 2 bytes de espaço

(B) Se o processador ignorar o salto, a próxima instrução seria 2 bytes à frente

Próxima Instrução = $40F780 + 2 = 40F782$

(C) somar o salto

Alvo = $40F782 + 3 = 40F785$

— 4 —

- (A) $0F85 \rightarrow$ é o opcode, apenas indica ao processador jmp
- (B) $F1FEFFFF$ ~~o~~ o valor do salto
- (C) Em x86, estão escritos em little-endian logo temos que ler do fim para o início juntando tudo, o número real ali escrito é $FFFFFFF1$
- (D) Como começa com F, este número é negativo
- (E) Para saber quanto é em decimal fazemos $0xFFFFFFFF - 0xFFFFFEF1$ e depois somamos 1
 $= 0x0000010E + 1 = 0x0000010F$
- (F) Próxima instrução é

$$8318A7 + 6 = 8318AF$$

$$\begin{aligned}\text{ALVO} &= 8318AF - 0x10F \\ &= 831798\end{aligned}$$

4.4) Endereço ALVO : 300834

Salto : 20

Tamanho da Instrução : 2 bytes

— 11 —

$$\begin{aligned}\text{Próxima instrução} &= 0x300834 - 0x20 \\ &= 0x300814\end{aligned}$$

$$\text{Endereço inicial} = 0x300814 - 2 = 0x300812$$

- Endereço ALUD : 854FA2
- Tamanho da instrução : 2 Bytes
- salto = E8

Como E é maior ou igual a 8, o salto é negativo :

$$(FF - E8) + 1$$

$$= 17 + 1 = 18$$

O salto é de - 0x18 bytes

Próxima instrução é 0x854FA2 + 0x18

$$= 0x854FAB$$

Endereço Inicial = 0x854FAB - 2

$$= 0x854FA9$$

4.5)

```
.comm a, 4, 4      ; int a
.comm b, 4, 4      ; int b
.comm c, 4, 4      ; int c

.LC0:
.string "%d"       ; formato scanf
.LC1:
.string "%d %d\n"  ; formato printf
.globl main
main:
    pushl $a        ; arg 2: endereço de a
    pushl $.LC0     ; arg 1: formato "%d"
    call scanf      ; scanf("%d", &a)
    addl $8, %esp    ; limpa 2 args da stack
    movl a, %eax     ; eax = a
    addl %eax, %eax  ; eax = eax + eax
    movl %eax, b     ; b = eax
    movl b, %eax     ; eax = b
    subl a, %eax     ; eax = eax - a
    movl %eax, c     ; c = eax
    pushl c         ; arg 3: c
    pushl b         ; arg 2: b
    pushl $.LC1     ; arg 1: formato
    call printf     ; printf("%d %d\n", b, c)
    addl $12, %esp   ; limpa 3 args da stack
    movl $0, %eax    ; return 0
    ret            ; fim da função
```

